

Navigators Technical Documentation

Navigators — Security Architecture & Threat Mitigation

Document Identifier: SEC-NAV-01
Status: Production Reference
Version: 1.0.0
Last Updated: 2026-09-23
Attribution: Developed by Navigators

1. Threat Model & Security Boundaries

Navigators manages safety-critical vehicular positioning and crowdsourced cartographic data. The security posture addresses four distinct threat vectors:

1. **Client-Side Tampering:** Manipulation of local JavaScript state, sensor feeds, or cached model weights on rooted/jailbroken devices.
 2. **Unauthorized Map Manipulation (Cartographic Vandalism):** Malicious actors attempting to inject false road closures, dangerous hazards, or fictitious POIs into the public live map.
 3. **Data Poisoning Attacks:** Malicious contributors submitting synthetic, corrupted, or inverted IMU telemetry to poison deep learning models during retraining.
 4. **Privilege Escalation & Insecure Direct Object References (IDOR):** Unauthenticated or regular users attempting to access administrative endpoints, approve their own submissions, or read other users' private location logs.
-

2. Cryptographic Controls & Session Protection

2.1 One-Way Token Hashing

- **Raw Bearer Tokens:** Generated using cryptographically secure random bytes (`secrets.token_urlsafe(32)`).
- **Zero Plaintext Token Storage:** Raw tokens are hashed immediately upon creation via SHA-256 (`hashlib.sha256(token.encode()).hexdigest()`). Only the hash is stored in the `sessions` table. A database leak cannot reveal active bearer credentials.
- **Session Revocation:** Logging out instantly sets `revoked_at = datetime('now')`, rendering the token permanently invalid.

2.2 Password Security

User passwords stored in `auth_identities` are hashed using industry-standard salted hashing algorithms (`hashlib.pbkdf2_hmac` / Argon2), preventing rainbow table attacks.

3. IDOR Prevention & Ownership Boundaries

Insecure Direct Object References (IDOR) are eliminated by enforcing ownership validation in the repository layer rather than relying on UI button hiding:

```
# Enforced in src/api/contributions.py
item = contrib_repo.get(contrib_id)
decision = authz_service.can(user=context, action="contribution:read", resource=item)
if not decision.allowed:
    raise HTTPException(status_code=403, detail=decision.reason)
```

- **Draft Privacy:** Draft contributions are completely invisible to other regular users. Calling `GET /api/v1/contributions/{id}` on another user's draft returns HTTP 403 Forbidden.
- **Listing Protection:** In `/api/v1/contributions` and `/api/v1/contributions/user/{id}`, drafts belonging to other accounts are automatically purged from the query results before response generation.
- **Self-Moderation Prevention:** A moderator attempting to approve their own contribution is rejected with `PERMISSION_DENIED`.

4. Privilege Escalation Prevention

1. **Immutable System Roles:** Roles cannot be modified by unprivileged users. Role assignment (`POST /api/v1/admin/users/{id}/roles`) requires the `user:manage_roles` permission, which is restricted strictly to `super_admin`.
2. **State Machine Invariants:** Critical lifecycle transitions (e.g., publishing a contribution to canonical live maps) cannot be triggered directly via CRUD endpoints. They must pass through `ContributionStateMachine.can_transition()`, which enforces source-state prerequisites and role requirements.

5. Immutable Security Auditing

All security-sensitive operations are recorded in the append-only `audit_logs` table: - Account logins and session revocations. - Role assignments and privilege changes. - Contribution approvals, rejections, and direct publications. - Dataset certifications and training job invocations. - Model candidate promotions and production rollbacks.

Each audit record captures `actor_id`, `action`, `resource_type`, `resource_id`, `old_state`, `new_state`, `ip_address`, and ISO-8601 UTC timestamp.

Developed by Navigators